

数据结构

笔记本： 专业课及面试

创建时间： 2021/6/17 21:37

更新时间： 2021/9/23 21:32

作者： 红豆糕

URL: <https://baike.baidu.com/item/%E6%95%B0%E6%8D%AE%E7%BB%93%E6%...>

- 1.有多少种数据结构?
- 2.hash有几种类型(即散列函数有几种类型)? 解决hash冲突的方法?
- 3.栈和队列的应用及区别? ✓
- 4.二叉树前序/中序/后序遍历的区别?
- 5.拓扑排序?
- 6.DFS、BFS描述?
- 7.如何编程判断一棵二叉树是完全二叉树?
- 8.如何高效地判断链表是否有环(Leetcode快慢指针)?
- 9.最小生成树(prim和kruscal)?
- 10.最短路径dijkstra?
- 15.B+树? B树和B+树的区别和用途?
- 16.红黑树?
- 17.如何用栈实现汉诺塔问题?
- 18.堆的实现与应用? 堆排序的数据结构是什么?
- 19.邻接表和邻接矩阵(如何存储大数据)? 画出邻接表的结构图?
- 20.广义表(head(tail(A)))?
- 21.广义表和线性表的区别和共同点?
- 22.给出一组数, $H(\text{key}) = \text{key} \bmod x$, 画出哈希表发生冲突时用线性再散列(开放定址法)、链地址法解决的过程?
- 23.给出一个序列, 写出快排、基数、冒泡、堆排.....的关键节点序列(经过一趟)(各种排序的手写结果必考! 随机出几种方法)?
- 24.给出一组序列, 分别画出二叉平衡树、二叉排序树最后插入后得到的图?
- 26.树以及图的存储方式?
- 27.哈夫曼编码?
- 28.递归的三个要素?
- 29.无向树?
- 30.哪些图算法中用到了动态规划的思想?
- 31.稀疏矩阵的三元组存储方式?
- 32.完全二叉树与哈夫曼树一样吗为什么?
- 34.如何通过栈, 将十进制整数n转换为N进制? 进制转换
- 35.如何递归求叶子结点的个数?
- 37.求一定范围内5或3的倍数, 但不是7的倍数?
- 38.堆和栈的区别?
- 39.二分查找?
- 40.你做算法时用到的什么数据结构比较好用?
- 41.描述一个自己最熟悉的算法和数据结构, 并谈谈两者之间的关联
- 43.给你一个单链表, 现在希望你能倒序输出, 你该怎么做?



- [44.给你两个有序链表，用什么样的方式将它们合并起来？](#)
- [45.讲一下你熟悉的排序和它们的时间复杂度？](#)
- [46.递归法、分治法、动态规划、贪心算法、回溯法、分支限界法？](#)

1.有多少种数据结构？

数据结构一共有8类：数组、栈、队列、链表、树、散列表、堆、图

- 数组：数组是可以在内存中连续存储多个元素的结构，在内存中的分配也是连续的，数组中的元素通过数组下标进行访问，数组下标从0开始。
- 栈：栈是一种特殊的线性表，仅能在线性表的一端操作，栈顶允许操作，栈底不允许操作。栈的特点是：先进后出；
- 队列：队列与栈一样，也是一种线性表，不同的是，队列可以在一端添加元素，在另一端取出元素，也就是：先进先出。
- 链表：链表是物理存储单元上非连续的、非顺序的存储结构，数据元素的逻辑顺序是通过链表的指针地址实现，每个元素包含两个结点，一个是存储元素的数据域(内存空间)，另一个是指向下一个结点地址的指针域。根据指针的指向，链表能形成不同的结构，例如单链表，双向链表，循环链表等。
- 树：它是由 n ($n \geq 1$) 个有限节点组成一个具有层次关系的集合。
- 散列表：散列表，也叫哈希表，是根据关键码和值(key和value)直接进行访问的数据结构，通过key和value来映射到集合中的一个位置，这样就可以很快找到集合中的对应元素。
- 堆：堆是一种比较特殊的数据结构，可以被看做一棵树的数组对象，具有以下性质：① 堆中某个节点的值总是不大于或不小于其父节点的值。② 堆总是一棵完全二叉树。将根节点最大的堆叫做大顶堆，根节点最小的堆叫做小顶堆。
- 图：图是由结点的有穷集合 V 和边的集合 E 组成。其中，为了与树形结构加以区别，在图结构中常常将结点称为顶点，边是顶点的有序偶对，若两个顶点之间存在一条边，就表示这两个顶点具有相邻关系。

2.hash有几种类型（即散列函数有几种类型）？解决hash冲突的方法？

散列函数有以下几种方法：直接定址法、除留余数法、数字分析法、平方取中法

7.4.2 散列函数的构造方法

在构造散列函数时，必须注意以下几点：

- 1) 散列函数的定义域必须包含全部需要存储的关键字，而值域的范围则依赖于散列表的大小或地址范围。
 - 2) 散列函数计算出来的地址应该能等概率、均匀地分布在整个地址空间中，从而减少冲突的发生。
 - 3) 散列函数应尽量简单，能够在较短的时间内计算出任一关键字对应的散列地址。
- 下面介绍常用的散列函数。

1. 直接定址法

直接取关键字的某个线性函数值为散列地址，散列函数为

$$H(\text{key}) = \text{key} \text{ 或 } H(\text{key}) = a \times \text{key} + b$$

式中， a 和 b 是常数。这种方法计算最简单，且不会产生冲突。它适合关键字的分布基本连续的

情况，若关键字分布不连续，空位较多，则会造成存储空间的浪费。

2. 除留余数法

这是一种最简单、最常用的方法，假定散列表表长为 m ，取一个不大于 m 但最接近或等于 m 的质数 p ，利用以下公式把关键字转换成散列地址。散列函数为

$$H(\text{key}) = \text{key} \% p$$

除留余数法的关键是选好 p ，使得每个关键字通过该函数转换后等概率地映射到散列空间上的任一地址，从而尽可能减少冲突的可能性。

3. 数字分析法

设关键字是 r 进制数（如十进制数），而 r 个数码在各位上出现的频率不一定相同，可能在某些位上分布均匀一些，每种数码出现的机会均等；而在某些位上分布不均匀，只有某几种数码经常出现，此时应选取数码分布较为均匀的若干位作为散列地址。这种方法适合于已知的关键字集合，若更换了关键字，则需要重新构造新的散列函数。

4. 平方取中法

顾名思义，这种方法取关键字的平方值的中间几位作为散列地址。具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀，适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数。

在不同的情况下，不同的散列函数具有不同的性能，因此不能笼统地说哪种散列函数最好。在实际选择中，采用何种构造散列函数的方法取决于关键字集合的情况，但目标是尽量降低产生冲突的可能性。

解决hash冲突的方法：

1. 开放定址法

所谓开放定址法，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为

$$H_i = (H(\text{key}) + d_i) \% m$$

式中， $H(\text{key})$ 为散列函数； $i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)； m 表示散列表表长； d_i 为增量序列。

取定某一增量序列后，对应的处理方法就是确定的。通常有以下 4 种取法：

- 1) 线性探测法。当 $d_i = 0, 1, 2, \dots, m - 1$ 时，称为线性探测法。这种方法的特点是：冲突发生时，顺序查看表中下一个单元（探测到表尾地址 $m - 1$ 时，下一个探测地址是表首地址 0），直到找出一个空闲单元（当表未填满时一定能找到一个空闲单元）或查遍全表。
线性探测法可能使第 i 个散列地址的同义词存入第 $i + 1$ 个散列地址，这样本应存入第 $i + 1$ 个散列地址的元素就争夺第 $i + 2$ 个散列地址的元素的地址……从而造成大量元素在相邻的散列地址上“聚集”（或堆积）起来，大大降低了查找效率。
- 2) 平方探测法。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 时，称为平方探测法，其中 $k \leq m/2$ ，散列表长度 m 必须是一个可以表示成 $4k + 3$ 的素数，又称二次探测法。

平方探测法是一种处理冲突的较好方法，可以避免出现“堆积”问题，它的缺点是不能探测到散列表上的所有单元，但至少能探测到一半单元。

- 3) 再散列法。当 $d_i = \text{Hash}_2(\text{key})$ 时，称为再散列法，又称双散列法。需要使用两个散列函数，当通过第一个散列函数 $H(\text{key})$ 得到的地址发生冲突时，则利用第二个散列函数 $\text{Hash}_2(\text{key})$ 计算该关键字的地址增量。它的具体散列函数形式如下：

$$H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key})) \% m$$

初始探测位置 $H_0 = H(\text{key}) \% m$ 。 i 是冲突的次数，初始为 0。在再散列法中，最多经过 $m - 1$ 次探测就会遍历表中所有位置，回到 H_0 位置。

- 4) 伪随机序列法。当 d_i 为伪随机数序列时，称为伪随机序列法。

注意：在开放定址的情形下，不能随便物理删除表中的已有元素，因为若删除元素，则会截断其他具有相同散列地址的元素的查找地址。因此，要删除一个元素时，可给它做一个删除标记，进行逻辑删除。但这样做的副作用是：执行多次删除后，表面上看起来散列表很满，实际上有许多位置未利用，因此需要定期维护散列表，要把删除标记的元素物理删除。

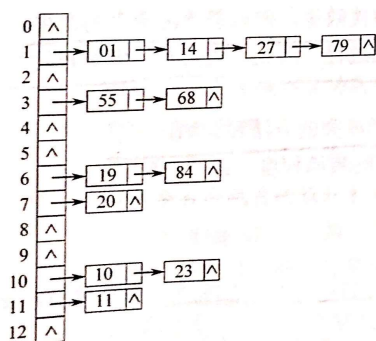


图 7.9 拉链法处理冲突的散列表

2. 拉链法（链接法，chaining）

显然，对于不同的关键字可能会通过散列函数映射到同一地址，为了避免非同义词发生冲突，可以把所有的同义词存储在一个线性链表中，这个线性链表由其散列地址唯一标识。假设散列地址为 i 的同义词链表的头指针存放在散列表的第 i 个单元中，因而查找、插入和删除操作主要在同义词链中进行。拉链法适用于经常进行插入和删除的情况。

例如，关键字序列为 {19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法处理冲突，建立的表如图 7.9 所示（学完下节内容后，可以尝试计算本例的平均查找长度 ASL）。

3. 栈和队列的应用及区别？

区别：

- ① 栈是先进后出，队列是先进先出；
- ② 对插入和删除操作的限制不同：栈是限制只能在表的一端进行插入或者删除的线性表，队列是限制只能在表的尾部进行插入而在表的头部进行删除的线性表；
- ③ 栈和队列遍历数据的速度不同。栈在遍历数据的时候需要为数据开辟临时空间，保持数据在遍历前的一致性。队列则不同，它是基于地址指针进行遍历，无需开辟临时空间，因为在遍历的过程中不影响数据结构，速度要快得多。

应用：

栈：括号匹配、表达式转换、迷宫问题；

队列：银行排队问题

4. 二叉树前序/中序/后序遍历的区别？

前序：根左右；

中序：左根右；

后序：左右根。

5. 拓扑排序？

什么是拓扑排序？

对一个有向无环图 G 进行拓扑排序，就是将图中所有顶点排成一个线性序列，使得图中任何一条有向边，假设有向边由 u 指向 v ，则线性序列中顶点 u 一定在顶点 v 之前，这

样的线性序列就称为拓扑序列。

如何得到拓扑序列？

先统计所有节点的入度，对于入度为0的节点就可以分离出来，然后把这个节点指向的节点的入度减一。重复这个操作，直到所有结点都被分离出来。

如果最后不存在入度为0的节点，那就说明有环，不存在拓扑排序

6.DFS、BFS描述？

深度优先搜索DFS：

深度优先搜索是一种枚举完所有路径以遍历所有情况的搜索方法。

使用深度优先搜索策略时，常常是为了知道问题是否有解，而一般不使用深度优先搜索求解最优解问题。

广度优先搜索：

广度优先搜索策略从搜索的起点开始，不断地优先访问当前结点的邻居。也就是说，首先访问起点，然后依次访问起点尚未访问的邻居结点，再按照访问起点邻居结点的优先顺序依次访问它们的邻居，直到找到解或搜遍整个解空间。

广度优先搜索常用来求解最优值问题。

7.如何编程判断一棵二叉树是完全二叉树？

根据完全二叉树的定义，如果二叉树上某个结点有右孩子而无左孩子，那么该二叉树一定不是完全二叉树；否则，如果该二叉树上某个结点有左孩子而无右孩子，那么在该结点所在那一层上，该结点右侧的所有结点都是叶子结点，否则就不是完全二叉树。

8.如何高效地判断链表是否有环（Leetcode快慢指针）？

9.最小生成树（prim和kruscal）？

生成树是指含有图的全部 n 个顶点，但只有足以构成一棵树的 $n-1$ 条边。

最小生成树是所有边的代价和最小的生成树。

Kruscal（克鲁斯卡尔算法）：

初始最小生成树的边数为0，每迭代依次就选择一条满足条件的最小代价边，加入到最小生成树的边集合中，直到所有顶点都在这棵树内。

Prim算法：

对于已经加入最小生成树的顶点集合，寻找这些顶点所有未被访问过的相邻顶点，找到其中的最小边，将该最小边对应的顶点加入最小生成树的顶点集合，直到所有的顶点都在这棵树内。

10.最短路径dijkstra？

dijkstra为单源最短路径算法，也就是求出从源点到其余各结点的最短路径。基本思想是：每次找到离源点最近的一个顶点，然后以该顶点为中心进行扩展，最终得到源点

到其余所有点的最短路径。

15.B+树？B树和B+树的区别和用途？

1. 什么是B树，B树的特点：

一棵m阶的B树或为空树，或是满足以下特性的m叉树：

- (1) 树中每个结点最多有m个子结点（子树），最多有m-1个关键字；
- (2) 除根节点外的所有非叶结点最少有 $m/2$ （向上取整）个子结点（子树），最少有 $m/2$ （向上取整）-1个关键字；
- (3) 若根节点不是终端结点（即叶节点的上一层结点），则至少有两棵子树；
- (4) 所有非叶节点都是带有多多个关键字的有序表，且都带有指向其子树根节点的指针；
- (5) 所有叶节点都出现在同一层上，并且不带信息（视为NULL结点）。

B树是所有结点的平衡因子均等于0的多路平衡二叉树。

2. 什么是B+树，B+树的特点：

一棵m阶的B+树需满足以下条件：

- (1) 每个分支结点最多有m个子结点/子树。--与B树一样
- (2) 结点的子树个数和关键字个数相等。--与B树不同
- (3) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $m/2$ （向上取整）棵子树。
- (4) 所有叶节点均带信息。叶节点包含所有非叶结点中出现的全部关键字以及指向相应记录的指针。叶节点中将关键字按大小顺序排列，并且相邻叶节点按大小顺序相互链接起来。
- (5) 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针。

3. B树与B+树的区别：

- (1) **在B+树中，具有n个关键字的结点只含有n棵子树**，即每个关键字对应一棵子树；而在B树中，具有n个关键字的结点有n+1棵子树。
- (2) 在B+树中，每个结点的**关键字的个数范围**和每个结点的子树个数范围一样；而在B树中，每个结点的关键字的个数范围比每个结点的子树个数范围-1。
- (3) 在**B+树中，叶节点包含信息**，所有非叶节点仅起到索引作用，非叶结点中每个索引项只含有对应子树的最大关键字和指向该子树的指针；而在**B树中，所有叶节点均不带信息（视为NULL结点）**。
- (4) 在**B+树中，叶节点包含了全部关键字**，即在非叶结点中出现的关键字也会出现在叶节点中；而在**B树中，终端结点（即叶节点的上一层）包含的关键字和其他结点包含的关键字是不重复的**。
- (5) B+树的叶节点之间需要**通过指针链接**；而B树不要求叶节点之间用指针链接。
- (6) B+树**支持顺序查找**，就像对顺序链表进行查找一样；而B树不支持顺序查找。

16.红黑树？

1. 定义：**红黑树是一种二叉查找树，但在每个节点都增加了一个存储位表示这个节点的颜色（红或黑）**。通过对任何一条从根到叶子的路径上各个节点着色的方式

的限制，**红黑树确保没有一条路径会比其它路径长出两倍**，因此，**红黑树是一种弱平衡二叉树**（由于是弱平衡，可以看到，在相同的结点情况下，AVL树的高度低于红黑树），相对于严格的AVL树（平衡二叉树）来说，红黑树的旋转次数少，所以对于搜索、插入、删除操作较多的情况下，我们就使用红黑树。

2. 性质：

红黑树是一棵自平衡（而不是绝对平衡）的二叉查找树（BST, Binary Search Tree）。

树上的每个结点都遵循下面的规则：

- ① 每个结点非红即黑；
- ② 根节点始终是黑色的；
- ③ 不能出现连续的红色结点（即任何一个红色结点的左右子女必须是黑色结点），但可以出现连续的黑色结点；
- ④ 从任何一个结点（包括根节点）到NULL结点（即叶子结点下方挂的两个空结点，并且认为这些NULL结点是黑色的）的每条路径都具有相同数量的黑色结点。

如下是一棵典型的红黑树：

正因为这些规则，才能保证红黑树的自平衡。**红黑树从根节点到叶子节点的最长路径不会超过最短路径的2倍。**

17.如何用栈实现汉诺塔问题？

利用递归的方法可以推出如下算法：

第一步 把A上的n-1个圆盘移到C上； 第二步 把A上的一个圆盘移到B上； 第三步 把C上的n-1个圆盘移到B上；

18.堆的实现与应用？堆排序的数据结构是什么？

堆排序是利用堆这种数据结构设计的一种排序算法

19.邻接表和邻接矩阵（如何存储大数据）？画出邻接表的结构图？

20.广义表（head(tail(A)))？

注意：一个广义表中的表头是第一个元素，表尾是除第一个元素之外的所有元素组成的新的广义表。

4.5.1 广义表的定义与性质

广义表的表尾总是广义表

广义表(generalized list)简称表,它是线性表的推广。一个广义表 LS 定义为 $n(n \geq 0)$ 个表元素 $\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_{n-1}$ 组成的有限序列。记作

$$LS = (\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_{n-1})$$

其中,LS 为表名。 $\alpha_i (0 \leq i \leq n-1)$ 是表中元素,它或者是数据元素(称为原子),或者是子表。 n 是表的长度,即表中元素的个数。表的长度不包括作为分界符的左括号“(”和右括号”)”和表元素之间的分隔符“,”。长度为 0 的表为空表。

习惯上,用大写字母表示表名,用小写字母表示原子元素。

如果 $n \geq 1$,则称广义表的第一个表元素 α_0 为广义表的表头(head),而由表中除 α_0 外其他元素组成的表 $(\alpha_1, \alpha_2, \dots, \alpha_{n-1})$ 称为广义表的表尾(tail)。

广义表的定义是递归的,因为在表的描述中又用到了表,允许表中有表。这种递归的定义能够很简洁地描述庞大而复杂的结构。下面给出一些广义表的例子。

(1) $A = ()$ 这是一个空表,表的长度为 0。它没有表头和表尾。

(2) $B = (a, b)$ 这是一个只包括原子的表,称为线性表。表的长度为 2。它的表头为 a ,表尾为 (b) 。表尾还是一个表,其表头为 b ,表尾是空表 $()$ 。

(3) $C = (c, (d, e, f))$ 这是一个长度为 2 的表,它的表头为 c ,表尾为 $((d, e, f))$ 。这个表尾仍然是表,其表头为 (d, e, f) ,表尾为空表 $()$ 。

(4) $D = (B, (C, A))$ 这是一个长度为 3 的表,它的 3 个表元素都是子表。其表头为 B ,表尾为 (C, A) 。

(5) $E = (B, D)$ 这是一个长度为 2 的表,表元素都是子表。表头为 B ,表尾为 (D) 。

(6) $F = (h, F)$ 这是一个长度为 2 的表,表头为 h ,表尾为 (F) ;这个表尾还是表,其表头为 F ,表尾为空表 $()$ 。对于表头来说,出现了递归。

由广义表定义,可以得到以下几个性质。

(1) 有次序性 在广义表中,各表元素在表中以线性序列排列,每个表元素至多一个直接前驱,一个直接后继。这个次序不能交换。

(2) 有长度 广义表中表元素个数一定,不能是无限的,可以是空表。

(3) 有深度 广义表的表元素可以是原表的子表,子表的表元素还可以是子表,……,因此广义表是多层次结构,图 4.24 用有根有序有向的树描述了广义表各元素之间的层次关系。表中括号的重数即为深度。在图 4.24 中,用圆形结点“○”表示表元素,用方形结点“□”表示原子元素。在层次结构顶端的“○”结点表示这个广义表。如果有名字的话,在结点

深度实际上就是广义表中括号的层数。

若广义表中自己是自己的子表 $\Rightarrow$ 递归 $\Rightarrow$ 深度无穷大。

21. 广义表和线性表的区别和共同点?

1. 广义表可以看成是线性表的推广,线性表是广义表的特例。
2. 线性表的元素仅限于原子项,若允许其元素具有自身结构,即构成广义表。
3. 广义表的结构相当灵活,在某种前提下,它可以兼容线性表、数组、树和有向图等各种常用的数据结构。
当二维数组的每行(或每列)作为子表处理时,二维数组即为一个广义表。
另外,树和有向图也可以用广义表来表示。

22. 给出一组数, $H(\text{key}) = \text{key} \bmod x$, 画出哈希表发生冲突时用线性再散列(开放定址法)、链地址法解决的过程?

23. 给出一个序列, 写出快排、基数、冒泡、堆排.....的关键节点序列(经过一趟)(各种排序的手写结果必考! 随机出几种方法)?

24.给出一组序列，分别画出二叉平衡树、二叉排序树最后插入后得到的图？

平衡二叉搜索树是一种结构平衡的二叉搜索树，即叶节点高度差的绝对值不超过1，并且左右两个子树都是一棵平衡二叉树。能在 $O(\log_2 n)$ 内完成插入、查找和删除操作，最早被发明的平衡二叉搜索树为AVL树。

平衡二叉树的构建（插入）：

1. 如果T为空时,则创建一个新结点;
2. 如果T不为空,判断是否存在相同的结点.如果二叉树中存在相同结点,则不需要插入;
3. 如果要插入的结点值e小于T的根结点值,则在T的左子树查找;
 - (1) 如果能在左子树中查找到,则不插入进去.返回False;
 - (2) 如果没有找到,则插入插入成功taller为TRUE,说明新结点e已经插入进去; 此时需要判断T的平衡因子;
 - (3) 如果出现LL结构的话，则需要进行左旋处理。
4. 如果要插入的结点的值大于T的根结点值,则在T的右子树查找;
 - (1) 如果能在右子树中查找到,则不插入进去.返回False;
 - (2) 如果没有找到,则插入插入成功taller为TRUE,说明新结点e已经插入进去;
 - (3) 如果出现RR结构，就要进行右旋处理。

26.树以及图的存储方式？

1. 树的存储方式有三种：双亲表示法、孩子表示法和孩子兄弟表示法
双亲表示法是在存储树的结点时，包括结点自身的值data和其双亲结点的下标。该存储结构在找某个结点的双亲时很容易，但是找某个结点的孩子结点时，最坏情况下需要访问整个数组。；
孩子表示法每个结点包含两个部分，第一个部分存储当前结点的值，另一部分用FirstChild域存储该结点第一个孩子结点的地址，然后用指针链串起来其他孩子的结点。但是这个方法要找结点的双亲结点很不容易；
孩子兄弟表示法：每个结点包含3个部分，即结点值data，指向该节点第一个孩子结点FirstChild，指向该结点的兄弟结点的NextSibling。
2. 图的存储方式有两种：邻接矩阵和邻接表
邻接矩阵：使用二维数组 $G[N][N]$ 存储图。如果 $G[i][j]$ 为1，则说明顶点i和顶点j之间有边；如果 $G[i][j]$ 为0，这说明不存在边。如果存在边权，则可以令 $G[i][j]$ 存放边权。适用于顶点数目不太大（一般不超过1000）的题目。
邻接表：每个顶点都可能若干条出边，如果把同一个顶点的所有出边放在一个列表中，那么N个顶点就会有N个列表（没有出边，则对应空表）。这N个列表称为图G的邻接表

27.哈夫曼编码？

28.递归的三个要素？

1. 递归函数要实现的功能;
2. 递归边界 (终止条件) ;
3. 递归表达式 (函数的等价关系式) 。

29.无向树?

连通而不含回路的无向图称为无向树，简称树。

30.哪些图算法中用到了动态规划的思想?

比如说最小生成树算法中的Kruskal算法。

31.稀疏矩阵的三元组存储方式?

三元组存储方式的策略是只存储非零元素。稀疏矩阵中每一个非零元素需由一个三元组 (i, j, a_{ij}) 唯一确定，三元组中的 i 就是行下标， j 是列下标， a_{ij} 是对应的元素值，稀疏矩阵中的所有非零元素构成三元组线性表。

32.完全二叉树与哈夫曼树一样吗为什么?

完全二叉树满足的条件是：如果二叉树的深度为 h ，除第 h 层外，其他各层结点的个数都达到了最大个数，第 h 层所有结点都连续集中在最左边，这就是完全二叉树。

而哈夫曼树则是给定 n 个权值作为 n 个叶节点，构造一棵完全二叉树，如果带权路径长度达到最小，则称这样的二叉树为哈夫曼树。哈夫曼树是带权路径长度最短的树，权值较大的结点离根较近。

34.如何通过栈，将十进制整数n转换为N进制?

通过短除法，将十进制数不断地被 N 整除，将得到的余数依次放入栈中，最后从栈顶开始取余数，按顺序得到的数就是转换成 N 进制的数。

35.如何递归求叶子结点的个数?

一个结点的左子树和右子树都为 `null` 就是叶子节点，我们需要对每个结点进行判断，依次遍历，遇到满足条件的进行计数即可。

37.求一定范围内5或3的倍数，但不是7的倍数?

只需要在对范围进行遍历的时候，保证当前数值取余5和3为0，取余7不为0。

38.堆和栈的区别？

1. 管理方式不同。堆的申请和释放工作则由程序员控制，容易产生内存泄露；栈由操作系统自动分配释放，不需要我们手动控制。
2. 生长方向不同。堆的内存地址由低到高，而栈的内存地址由高到低。
3. 分配方式不同。堆都是动态分配的，由`allocate`函数进行分配；而栈有两种分配方式，分别是静态分配和动态分配，静态分配由操作系统完成，比如局部变量的分配，而动态分配则由操作系统进行释放，不需要我们手动实现。

39.二分查找？

二分查找适用于有序数列，先确定区间的中间位置，如果要查找的值比这个位置的数要大，就往右边查找，否则就往左边查找。

时间复杂度计算：最坏情况是数据长度为1的时候才找到那个数据，也就是 $N/2^k=1$ ，那么查找次数 $k=\log_2 n$ ，所以时间复杂度就是 $O(\log_2 n)$ 。

40.你做算法时用到的什么数据结构比较好用？

我觉得比较好用的数据结构有队列，比如说可以使用队列来实现广度优先搜索，对于当前的扩展结点，每次将它们的邻居结点都放入队列中，再依次从队列中取出结点进行遍历，直到遍历完所有可能性。

41.描述一个自己最熟悉的算法和数据结构，并谈谈两者之间的关联

比如说对于回溯法和分枝限界法，使用的数据结构就是树。

43.给你一个单链表，现在希望你能倒序输出，你该怎么做？

首先对单链表进行遍历，将链表中的元素依次放入栈中。这样，链表前面的元素在栈底，后面的元素在栈顶，就可以实现链表的倒序输出。

44.给你两个有序链表，用什么样的方式将它们合并起来？

采用双指针的方式。

- ① 首先将两个链表的头结点进行比较，小的那个作为合并后的链表的头结点。
- ② 同时，创建两个指针分别指向两个链表的头结点或头结点的下一个结点。
- ③ 当两个链表当前进行比较的元素都不为空，就依次从小到大连接；如果有一个链表中的结点开始为空，则将另一个链表的剩余结点添加到合并链表的末尾。

45.讲一下你熟悉的排序和它们的时间复杂度？

冒泡排序、选择排序、插入排序： $O(N^2)$ ；

归并排序、快排、堆排： $O(N\log N)$ ；

桶排： $O(N)$

46.递归法、分治法、动态规划、贪心算法、回溯法、分支限界法？

1. 递归法：直接或间接调用其自身的算法就是递归法（例如跳台阶问题、斐波那契数列、汉诺塔问题）；
2. 分治法：将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破、分而治之；
3. 动态规划：将待求解的问题分解成若干个相互联系的子问题，先求解子问题，然后从这些子问题的解得到原问题的解；对于重复遇见的子问题，只在第一次遇见的时候对它进行求解，并把答案保存起来，让以后每次遇到时直接引用答案，不必重新求解。动态规划算法最关键的特征就是考虑问题是否具有最优子结构性质（比如矩阵连乘问题、最长公共子序列问题）；
4. 贪心算法：贪心算法采用逐步构造最优解的方法。在每个阶段，都做出一个看上去最优的选择。它与动态规划算法的区别在于贪心算法依赖于以往做出的选择，不依赖于子问题的解，而动态规划算法，每步所作的选择依赖于相关子问题的解（比如时间安排问题、背包问题）。
5. 回溯法（深度优先）：基本思想是从一条路往前走，能进则进，不能进则退回来，换一条路再试。
6. 分支限界法（广度优先）：分支限界法常以广度优先或以最小耗费有限的方式搜索问题的解空间树。

在搜索问题的解空间树时，分支限界法和回溯法的主要区别在于它们对当前扩展节点所采用的扩展方式不同。在分支限界法中，每一个活结点只有一次机会成为扩展节点。活结点一旦成为扩展节点，就一次性产生其所有儿子节点。在这些儿子节点中，导致不可行解或导致非最优解的儿子节点被舍弃，其余儿子节点被加入活结点表中。此后，从活结点表中取下一节点为当前扩展节点。并重复上述节点扩展过程。这个过程移至持续到找到所需的解或活结点表为空为止。

《数据结构》笔记

第一章 绪论

1.什么是数据结构？

数据结构是数据对象在计算机中的组织方式。这样的组织方式包括逻辑结构（例如线性结构、树、图等）以及物理存储结构（在内存中的存放方法，用数组来存还是用链表来存？）。

数据结构是指相互之间存在一种或多种特定关系的数据元素的集合。通常情况下，精心选择的数据结构可以带来更高的运行或者存储效率。

第七章 查找

《王道》P261

1.什么是平均查找长度？

在查找过程中，一次查找的长度是指需要比较的关键字次数，而平均查找长度则是所有查找过程中进行关键字的比较次数的平均值。其数学定义为：

$$ASL = \sum_{i=1}^n P_i C_i$$

式中，n是查找表的长度，Pi是查找第i个数据元素的概率，一般认为每个数据元素的查找概率相等。即Pi=1/n；Ci是找到第i个数据元素所需进行的比较次数。**平均查找长度是衡量查找算法效率的最主要指标。**

2.顺序查找

顺序查找又称为线性查找，对顺序表和链表都是适用的。对于顺序表，可以通过数组下标递增来顺序扫描每个元素；对于链表，可通过指针next来依次扫描每个元素。顺序查找通常分为对一般的无序线性表的顺序查找和对有序线性表的顺序查找。

(1) 无序线性表的顺序查找

基本思想：从线性表的一端开始，逐个检查关键字是否满足给定的条件。
查找成功时，顺序查找的平均查找长度为（假设每个元素的查找概率相等）：

$$ASL_{success} = \sum_{i=1}^n P_i (n - i + 1) = \frac{n + 1}{2}$$

查找不成功时，平均查找长度为：

$$ASL_{unsuccess} = n + 1$$

(2) 有序表的顺序查找

若在查找之前就已经知道表是关键字有序的，则查找失败时可以不用再比较到表的另一端就能返回查找失败的信息，从而降低顺序查找失败的平均查找长度。
可以使用判定树来描述有序线性表的查找过程：

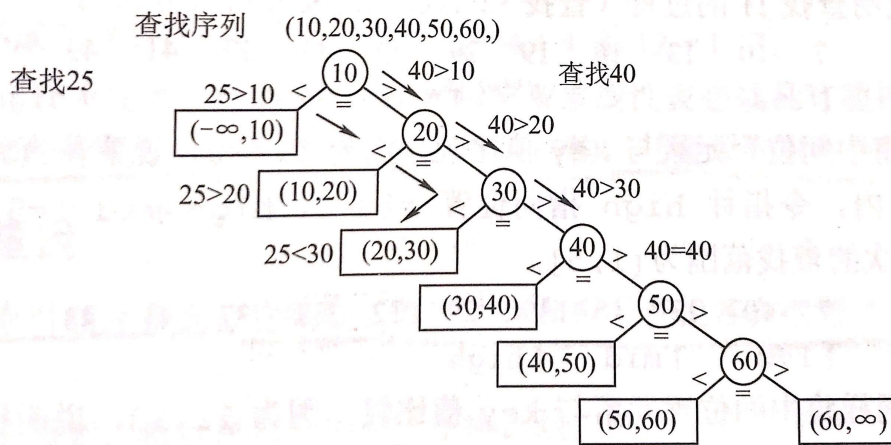


图 7.1 有序顺序表上的顺序查找判定树

查找成功的平均查找长度与无序线性表的顺序查找一样：

$$ASL_{success} = \sum_{i=1}^n P_i(n-i+1) = \frac{n+1}{2}$$

查找失败时，到达失败结点时所查找的长度等于它上面的一个圆形结点的所在层数。则查找失败的平均查找长度为：

$$ASL_{unsuccess} = \sum_{j=1}^n q_j(l_j - 1) = \frac{1+2+\dots+n+n}{n+1} = \frac{n}{2} + \frac{n}{n+1}$$

式中， q_j 是到达第 j 个失败结点的概率（一共 $n-1$ 个失败结点）； l_j 是第 j 个失败结点所在的层数。

3.折半查找

7.2.2 折半查找

折半查找又称二分查找，它仅适用于有序的顺序表。

折半查找的基本思想：首先将给定值 key 与表中中间位置的元素比较，若相等，则查找成功，返回该元素的存储位置；若不等，则所需查找的元素只能在中间元素以外的前半部分或后半部分（例如，在查找表升序排列时，若给定值 key 大于中间元素，则所查找的元素只可能在后半部分）。然后在缩小的范围内继续进行同样的查找，如此重复，直到找到为止，或确定表中没有所需要查找的元素，则查找不成功，返回查找失败的信息。算法如下：

```
int Binary_Search(SeqList L, ElemType key) {
    int low=0, high=L.TableLen-1, mid;
    while(low<=high) {
        mid=(low+high)/2;           //取中间位置
        if(L.elem[mid]==key)
            return mid;             //查找成功则返回所在位置
        else if(L.elem[mid]>key)
            high=mid-1;             //从前半部分继续查找
        else
            low=mid+1;              //从后半部分继续查找
    }
    return -1;                     //查找失败，返回-1
}
```

可以使用判定树描述查找过程，判定树是一棵平衡二叉树：

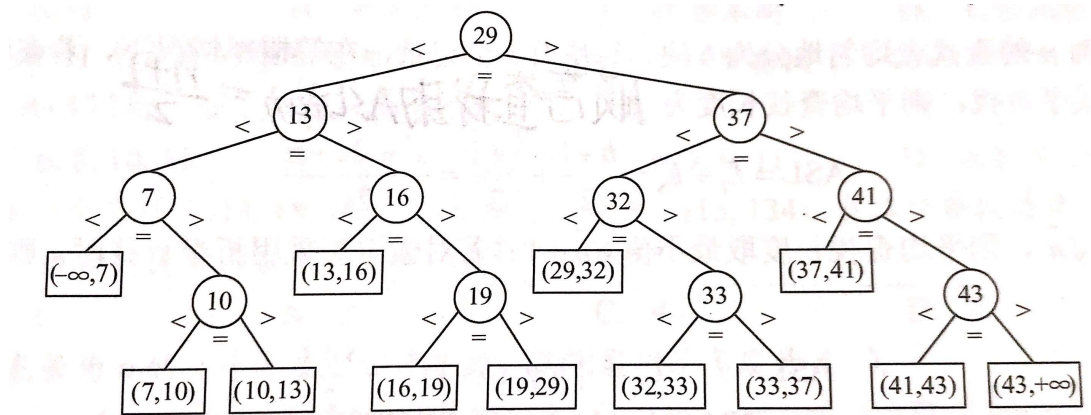


图 7.2 描述折半查找过程的判定树

从判定树可以看出，查找成功时的查找长度为从根节点到目的结点的路径上的结点数，而查找不成功时的查找长度为从根节点到对应失败结点的父结点的路径上的结点数。

查找成功的平均查找长度为：

$$ASL_{success} = \frac{1}{n} \sum_{i=1}^n l_i = \frac{1}{n} (1 \times 1 + 2 \times 2 + 3 \times 4 + \dots + h \times 2^{(h-1)}) = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

折半查找的时间复杂度为 $O(\log_2 n)$ 。

4.分块查找

1. 分块查找又称为索引顺序查找。它吸取了顺序查找和折半查找各自的优点。
2. 基本思想：将查找表分为若干子块，块内的元素可以无序，但块之间是有序的（即前一个块中的最大关键字小于后一个块中的所有关键字）。
3. 分块查找的过程：
 - (1) 先在索引表中确定待查记录所在的块（顺序查找或折半查找）；
 - (2) 在块内顺序查找。
4. 分块查找的平均查找长度为索引查找和块内查找对平均长度之和。设索引查找和块内查找的平均查找长度分别为 L_I 和 L_S ，则分块查找的平均查找长度为：
将长度为 n 的查找表均匀地分为 b 块，每块有 s 个记录，在等概率情况下，若在**块内和索引表中均采用顺序查找**，则平均查找长度为：

$$ASL_{success} = L_I + L_S = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2s + n}{2s}$$

若索引表中采取折半查找，则平均查找长度为：

$$ASL_{success} = L_I + L_S = \lceil \log_2(b+1) \rceil + \frac{s+1}{2}$$

5.B树

1. B树的概念：B树，又称多路平衡查找树，B树中所有结点的孩子个数的最大值为B树的阶，通常用 m 表示，即每一个结点的子结点最多有 m 个。
2. 一棵 m 阶的B树或为空树，或是满足以下特性的 m 叉树：
 - (1) 树中每个结点最多有 m 个子结点（子树），最多有 $m-1$ 个关键字；
 - (2) 除根节点外的所有非叶结点最少有 $m/2$ （向上取整）个子结点（子树），最少有 $m/2$ （向上取整）-1个关键字；

- (3) 若根节点不是终端结点（即叶节点的上一层结点），则至少有两棵子树；
 - (4) 所有非叶节点都是带有多个关键字的有序表，且都带有指向其子树根节点的指针；
 - (5) 所有叶节点都出现在同一层上，并且不带信息（视为NULL结点）。
- B树是所有结点的平衡因子均等于0的多路平衡二叉树。

3. 如下是一个5阶B树：

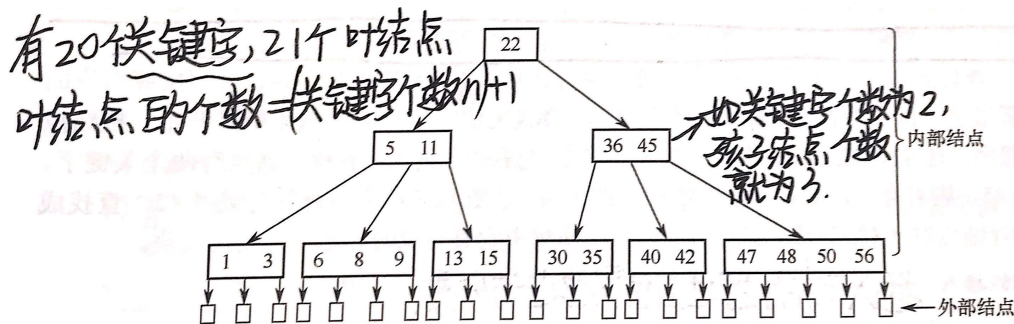


图 7.4 一棵 5 阶 B 树的实例

- (1) 叶节点的个数 $21 = B$ 树上所有结点关键字的个数 $20 + 1$;
- (2) 每个结点的孩子个数等于该结点中的关键字个数 $+ 1$;
- (3) 除根节点外所有非终端结点的子树个数的范围：

$$[\lceil \frac{m}{2} \rceil, m]$$

除根节点外所有非终端结点的关键字个数的范围：

$$[\lceil \frac{m}{2} \rceil - 1, m - 1]$$

- (4) 结点中关键字从左到右递增有序，每一个关键字两侧均有指向子树的指针，左边指针所指子树的所有关键字均小于该关键字，右边指针所指子树的所有关键字均大于该关键字。

4. B树的基本操作：B树的高度、B树的查找、插入、删除

1. B 树的高度（磁盘存取次数）

由下一节将得知，B 树中的大部分操作所需的磁盘存取次数与 B 树的高度成正比。

下面来分析 B 树在不同情况下的高度。当然，首先应该明确 B 树的高度不包括最后的不带任何信息的叶结点所处的那一层（有些书对 B 树的高度的定义中，包含最后的那一层）。

若 $n \geq 1$ ，则对任意一棵包含 n 个关键字、高度为 h 、阶数为 m 的 B 树：

- (1) 因为 B 树中每个结点最多有 m 棵子树， $m - 1$ 个关键字，所以在一棵高度为 h 的 m 阶 B 树中关键字的个数应满足 $n \leq (m - 1)(1 + m + m^2 + \dots + m^{h-1}) = m^h - 1$ ，因此有

$$h \geq \log_m(n + 1)$$

- (2) 若让每个结点中的关键字数达到最少，则容纳同样多关键字的 B 树的高度达到最大。

由 B 树的定义：第一层至少有 1 个结点；第二层至少有 2 个结点；除根结点外的每个非终端结点至少有 $\lceil m/2 \rceil$ 棵子树，则第三层至少有 $2\lceil m/2 \rceil$ 个结点……第 $h + 1$ 层至少有 $2(\lceil m/2 \rceil)^{h-1}$ 个结点，注意到第 $h + 1$ 层是不包含任何信息的叶结点。对于关键字个数为 n 的 B 树，叶结点即查找不成功的结点为 $n + 1$ ，由此有 $n + 1 \geq 2(\lceil m/2 \rceil)^{h-1}$ ，即 $h \leq \log_{\lceil m/2 \rceil}(n + 1)$

因为第 2 层至少有 2 个结点，
对于所有 m 阶的 h 的 B 树最少有 $2(\lceil m/2 \rceil)^{h-1}$ 个叶结点

$$((n+1)/2)+1, m=3, n=8$$

例如，假设一棵 3 阶 B 树共有 8 个关键字，则其高度范围为 $2 \leq h \leq 3.17$ 。

2. B 树的查找 B 树插入支持顺序查找

在 B 树上进行查找与二叉查找树很相似，只是每个结点都是多个关键字的有序表，在每个结点上所做的不是两路分支决定，而是根据该结点的子树所做的多路分支决定。

B 树的查找包含两个基本操作：① 在 B 树中找结点；② 在结点内找关键字。由于 B 树常存储在磁盘上，因此前一个查找操作是在磁盘上进行的，而后一个查找操作是在内存中进行的，即在找到目标结点后，先将结点信息读入内存，然后在结点内采用顺序查找法或折半查找法。

在 B 树上查找某个结点后，先在有序表中进行查找，若找到则查找成功，否则按照对应的指针信息到所指的子树中去查找（例如，在图 7.4 中查找关键字 42，首先从根结点开始，根结点只有一个关键字，且 $42 > 22$ ，若存在，必在关键字 22 的右边子树上，右孩子结点有两个关键字，而 $36 < 42 < 45$ ，则若存在，必在 36 和 45 中间子树上，在该子结点中查到关键字 42，查找成功）。查找到叶结点时（对应指针为空指针），则说明树中没有对应的关键字，查找失败。

3. B 树的插入 插入元素后超过上限需要进行分裂

与二叉查找树的插入操作相比，B 树的插入操作要复杂得多。在二叉查找树中，仅需查找到需插入的终端结点的位置。但是，在 B 树中找到插入的位置后，并不能简单地将其添加到终端结点中，因为此时可能会导致整棵树不再满足 B 树定义中的要求。将关键字 key 插入 B 树的过程如下：

- 1) 定位。利用前述的 B 树查找算法，找出插入该关键字的最低层中的某个非叶结点（在 B 树中查找 key 时，会找到表示查找失败的叶结点，这样就确定了最低层非叶结点的插入位置。注意：插入位置一定是最低层中的某个非叶结点）。

- 2) 插入。在 B 树中，每个非失败结点的关键字个数都在区间 $[m/2]-1, m-1$ 内。插入后的结点关键字个数小于 m ，可以直接插入；插入后检查被插入结点内关键字的个数，当插入后的结点关键字个数大于 $m-1$ 时，必须对结点进行分裂。

分裂的方法是：取一个新结点，在插入 key 后的原结点，从中间位置 ($\lceil m/2 \rceil$) 将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置 ($\lceil m/2 \rceil$) 的结点插入原结点的父结点。若此时导致其父结点的关键字个数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致 B 树高度增 1。

对于 $m=3$ 的 B 树，所有结点中最多有 $m-1=2$ 个关键字，若某结点中已有两个关键字，则结点已满，如图 7.5(a) 所示。插入一个关键字 60 后，结点内的关键字个数超过了 $m-1$ ，如图 7.5(b) 所示，此时必须进行结点分裂，分裂的结果如图 7.5(c) 所示。

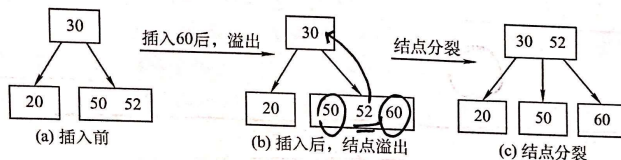


图 7.5 结点的“分裂”示意

4. B 树的删除

删除元素后低于下限需要进行合并 在非终端结点
在终端结点

B 树中的删除操作与插入操作类似，但要稍微复杂一些，即要使得删除后的结点中的关键字个数 $\geq \lceil m/2 \rceil - 1$ ，因此将涉及结点的“合并”问题。

即叶结点的上层.

当被删关键字 k 不在终端结点（最低层非叶结点）中时，可以用 k 的前驱（或后继） k' 来替代 k ，然后在相应的结点中删除 k' ，关键字 k' 必定落在某个终端结点中，则转换成了被删关键字在终端结点中的情形。在图 7.6 的 4 阶 B 树中，删除关键字 80，用其前驱 78 替代，然后在终端结点中删除 78。因此只需讨论删除终端结点中关键字的情形。

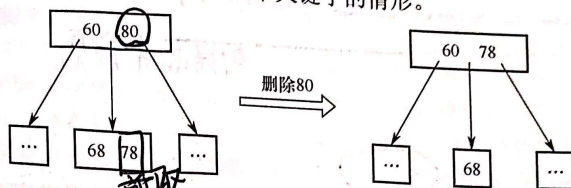
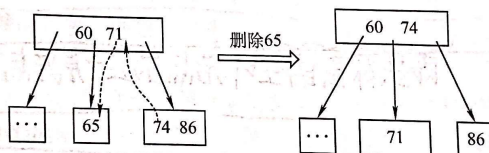


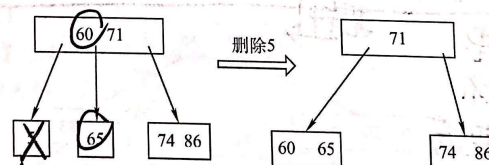
图 7.6 B 树中删除非终端结点关键字的取代

当被删关键字在终端结点（最低层非叶结点）中时，有下列三种情况：

- 1) 直接删除关键字。若被删除关键字所在结点的关键字个数 $\geq \lceil m/2 \rceil$ ，表明删除该关键字后仍满足 B 树的定义，则直接删去该关键字。
- 2) 兄弟够借。若被删除关键字所在结点删除前的关键字个数 $= \lceil m/2 \rceil - 1$ ，且与此结点相邻的右（或左）兄弟结点的关键字个数 $\geq \lceil m/2 \rceil$ ，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法），以达到新的平衡。在图 7.7(a) 中删除 4 阶 B 树的关键字 65，右兄弟关键字个数 $\geq \lceil m/2 \rceil = 2$ ，将 71 取代原 65 的位置，将 74 调整到 71 的位置。



(a) 兄弟够借



(b) 兄弟不够借

5. 删除 5 还要保证它有 3 个子树 (关键字个数 + 1)!

- 3) 兄弟不够借。若被删除关键字所在结点删除前的关键字个数 $= \lceil m/2 \rceil - 1$ ，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并。在图 7.7(b) 中删除 4 阶 B 树的关键字 5，它及其右兄弟结点的关键字个数 $= \lceil m/2 \rceil - 1 = 1$ ，故在 5 删除后将 60 合并到 65 结点中。

在合并过程中，双亲结点中的关键字个数会减 1。若其双亲结点是根结点且关键字个数减少至 0（根结点关键字个数为 1 时，有 2 棵子树），则直接将根结点删除，合并后的新结点成为根；若双亲结点不是根结点，且关键字个数减少到 $\lceil m/2 \rceil - 2$ ，则又要与它自己的兄弟结点进行调整或合并操作，并重复上述步骤，直至符合 B 树的要求为止。

6. B+ 树

1. B+ 树是应数据库所需而出现的一种 B 树的变形树。

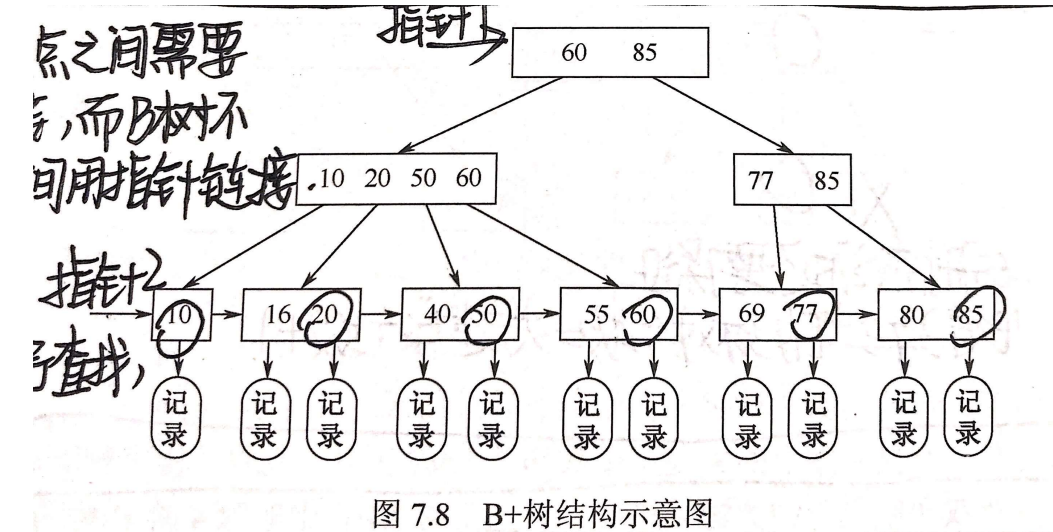
2. 一棵 m 阶的 B+ 树需满足以下条件：

- (1) 每个分支结点最多有 m 个子结点/子树。--与 B 树一样
- (2) 结点的子树个数和关键字个数相等。--与 B 树不同
- (3) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $m/2$ （向上取整）棵子树。

(4) 所有叶节点均带信息。叶节点包含所有非叶结点中出现的全部关键字以及指向相应记录的指针。叶节点中将关键字按大小顺序排列，并且相邻叶节点按大小顺序相互链接起来。

(5) 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针。

3. 如下所示就是一棵B+树：



7.B树与B+树的区别

1. 在B+树中，具有n个关键字的结点只含有n棵子树，即每个关键字对应一棵子树；而在B树中，具有n个关键字的结点有n+1棵子树。
2. 在B+树中，每个结点的关键字的个数范围和每个结点的子树个数范围一样；而在B树中，每个结点的关键字的个数范围比每个结点的子树个数范围-1。
3. 在B+树中，叶节点包含信息，所有非叶节点仅起到索引作用，非叶结点中每个索引项只含有对应子树的最大关键字和指向该子树的指针；而在B树中，所有叶节点均不带信息（视为NULL结点）。
4. 在B+树中，叶节点包含了全部关键字，即在非叶结点中出现的关键字也会出现在叶节点中；而在B树中，终端结点（即叶节点的上一层）包含的关键字和其他结点包含的关键字是不重复的。
5. B+树的叶节点之间需要通过指针链接；而B树不要求叶节点之间用指针链接。
6. B+树支持顺序查找，就像对顺序链表进行查找一样；而B树不支持顺序查找。

8.散列表

1. 散列函数：一个把查找表中的关键字映射成该关键字对应的地址的函数，记为 $\text{Hash}(\text{key}) = \text{Addr}$ 。
散列表：散列表建立了关键字和存储地址之间的一种映射关系。

2. 散列函数的构造方法:

7.4.2 散列函数的构造方法

在构造散列函数时, 必须注意以下几点:

- 1) 散列函数的定义域必须包含全部需要存储的关键字, 而值域的范围则依赖于散列表的大小或地址范围。
- 2) 散列函数计算出来的地址应该能等概率、均匀地分布在整个地址空间中, 从而减少冲突的发生。
- 3) 散列函数应尽量简单, 能够在较短的时间内计算出任一关键字对应的散列地址。

下面介绍常用的散列函数。

1. 直接定址法

直接取关键字的某个线性函数值为散列地址, 散列函数为

$$H(\text{key}) = \text{key} \text{ 或 } H(\text{key}) = a \times \text{key} + b$$

式中, a 和 b 是常数。这种方法计算最简单, 且不会产生冲突。它适合关键字的分布基本连续的情况, 若关键字分布不连续, 空位较多, 则会造成存储空间的浪费。

2. 除留余数法

这是一种最简单、最常用的方法, 假定散列表表长为 m , 取一个不大于 m 但最接近或等于 m 的质数 p , 利用以下公式把关键字转换成散列地址。散列函数为

$$H(\text{key}) = \text{key} \% p$$

除留余数法的关键是选好 p , 使得每个关键字通过该函数转换后等概率地映射到散列空间上的任一地址, 从而尽可能减少冲突的可能性。

3. 数字分析法

设关键字是 r 进制数 (如十进制数), 而 r 个数码在各位上出现的频率不一定相同, 可能在某些位上分布均匀一些, 每种数码出现的机会均等; 而在某些位上分布不均匀, 只有某几种数码经常出现, 此时应选取数码分布较为均匀的若干位作为散列地址。这种方法适合于已知的关键字集合, 若更换了关键字, 则需要重新构造新的散列函数。

4. 平方取中法

顾名思义, 这种方法取关键字的平方值的中间几位作为散列地址。具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系, 因此使得散列地址分布比较均匀, 适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数。

在不同的情况下, 不同的散列函数具有不同的性能, 因此不能笼统地说哪种散列函数最好。在实际选择中, 采用何种构造散列函数的方法取决于关键字集合的情况, 但目标是尽量降低产生冲突的可能性。

3. 处理冲突的方法:

1. 开放定址法

所谓开放定址法, 是指可存放新表项的空闲地址既向它的同义词表项开放, 又向它的非同义词表项开放。其数学递推公式为

$$H_i = (H(\text{key}) + d_i) \% m$$

式中, $H(\text{key})$ 为散列函数; $i = 0, 1, 2, \dots, k$ ($k \leq m - 1$); m 表示散列表表长; d_i 为增量序列。

取定某一增量序列后, 对应的处理方法就是确定的。通常有以下 4 种取法:

- 1) 线性探测法。当 $d_i = 0, 1, 2, \dots, m - 1$ 时, 称为线性探测法。这种方法的特点是: 冲突发生时, 顺序查看表中下一个单元 (探测到表尾地址 $m - 1$ 时, 下一个探测地址是表首地址 0), 直到找出一个空闲单元 (当表未填满时一定能找到一个空闲单元) 或查遍全表。
线性探测法可能使第 i 个散列地址的同义词存入第 $i + 1$ 个散列地址, 这样本应存入第 $i + 1$ 个散列地址的元素就争夺第 $i + 2$ 个散列地址的元素的地址……从而造成大量元素在相邻的散列地址上“聚集” (或堆积) 起来, 大大降低了查找效率。
- 2) 平方探测法。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 时, 称为平方探测法, 其中 $k \leq m/2$, 散列表长度 m 必须是一个可以表示成 $4k + 3$ 的素数, 又称二次探测法。

平方探测法是一种处理冲突的较好方法，可以避免出现“堆积”问题，它的缺点是不能探测到散列表上的所有单元，但至少能探测到一半单元。

- 3) 再散列法。当 $d_i = \text{Hash}_2(\text{key})$ 时，称为再散列法，又称双散列法。需要使用两个散列函数，当通过第一个散列函数 $H(\text{key})$ 得到的地址发生冲突时，则利用第二个散列函数 $\text{Hash}_2(\text{key})$ 计算该关键字的地址增量。它的具体散列函数形式如下：

$$H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key})) \% m$$

初始探测位置 $H_0 = H(\text{key}) \% m$ 。i 是冲突的次数，初始为 0。在再散列法中，最多经过 $m - 1$ 次探测就会遍历表中所有位置，回到 H_0 位置。

- 4) 伪随机序列法。当 d_i 为伪随机数序列时，称为伪随机序列法。

注意：在开放定址的情形下，不能随便物理删除表中的已有元素，因为若删除元素，则会截断其他具有相同散列地址的元素的查找地址。因此，要删除一个元素时，可给它做一个删除标记，进行逻辑删除。但这样做的副作用是：执行多次删除后，表面上看起来散列表很满，实际上有许多位置未利用，因此需要定期维护散列表，要把删除标记的元素物理删除。

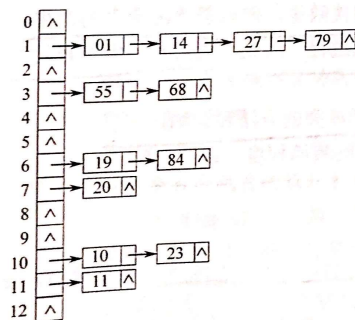


图 7.9 拉链法处理冲突的散列表

2. 拉链法 (链接法, chaining)

显然，对于不同的关键字可能会通过散列函数映射到同一地址，为了避免非同义词发生冲突，可以把所有的同义词存储在一个线性链表中，这个线性链表由其散列地址唯一标识。假设散列地址为 i 的同义词链表的头指针存放在散列表的第 i 个单元中，因而查找、插入和删除操作主要在同义词链中进行。拉链法适用于经常进行插入和删除的情况。

例如，关键字序列为 {19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法处理冲突，建立的表如图 7.9 所示（学完下节内容后，可以尝试计算本例的平均查找长度 ASL）。

n. 知识点集合

1. 递归在很多情况下会出现内存空间无法承受的情况，所以有的时候，对于数据量较大的情况，使用普通算法会比递归更好。